ml 1 report 3

paul.kothgasser

May 2026

Contents

1 Problem Description **2**
1.1 AI usage . . . 3

2 Exploratory Data Analysis **4**
2.1 Figures . . . 4
2.2 Explanation . . . 4
2.2.1 Boxplots (Figure 1) . . . 4
2.2.2 Feature Mean Heatmap (Figure 2) . . . 11
2.2.3 Correlation Matrix (Figure 3) . . . 11
2.2.4 Label Frequency (Figure 4) . . . 11
2.2.5 PCA Weights (Figure 5) . . . 11
2.2.6 Feature Comparisons (Figure 6) . . . 11
2.2.7 Scatter Plots (Figure 7) . . . 11

3 Baseline Model **12**
3.1 Validation Strategy . . . 12
3.2 Preprocessing . . . 12
3.3 Choice of K . . . 13
3.4 Preventing Data Leakage . . . 13

4 Model Experimentation and Validation **13**
4.1 Experimental Setup . . . 14
4.2 Feature Engineering Attempts . . . 15
4.2.1 Polynomial Features for Feature 10 . . . 15
4.2.2 Interaction Features . . . 15
4.2.3 Outlier Removal via Cluster Analysis . . . 15
4.3 Handling Class Imbalance . . . 16
4.4 Data Preprocessing . . . 16
4.5 Complete Experiment Results . . . 16
4.5.1 KNN Experiments . . . 16
4.6 Balanced Random Forest Refinement . . . 25
4.6.1 Final Model Choice . . . 25
4.7 Summary of Experimental Findings . . . 26

4.7.1 Effective Approaches 26
4.7.2 Ineffective Approaches 26
4.8 Confusion Matrix Analysis 27
4.9 Lessons Learned 29

5 Outlier detection **29**
5.1 Strategy Overview 29
5.2 Gaussian Mixture Model Training 30
5.3 Threshold Selection 30
5.4 Results 30
5.5 Performance Comparison 31
5.6 Discussion 31
5.7 Final Outlier-Aware Model: Optimod 31

6 Model Explainability **32**
6.1 Permutation Feature Importance (PFI) 32
6.1.1 Limitations 32
6.2 SHAP (SHapley Additive exPlanations) 33
6.2.1 Limitations 33
6.3 LIME (Local Interpretable Model-agnostic Explanations) 34
6.3.1 Limitations 34
6.4 Comparison and Interpretation 36
6.5 Key Observations 36
6.5.1 Disagreements Between Methods 36
6.5.2 Agreements Between Methods 37
6.6 Interpretation for Business Stakeholders 37

1 Problem Description

You work in a company that produces sensors that measure several physical quantities at the production sites of your customers. Your company has developed a sensor that can detect if a certain discrete event y ∈ [0, 1, 2, 3, 4] occurs at a production facility of your customer. Moreover, the sensor records auxiliary information about the environment (scalar values, collected in x ∈ R^d , d = 12) when it reports the event y. However, the parts of the sensor that "directly" measure y are extremely expensive, whereas the parts that measure the auxiliary information x are much cheaper. The company also assumes that there is some relationship between x and y, which you are tasked to find. If there was a model fθ that predicts y from x well, your company could omit the expensive parts of the sensor, just record x and predict y using fθ . Not all classes occur equally often, but they are considered to be equally important. Data Collection. To collect data, the company intern is tasked to mount one sensor at each of the three different production sites at your customer's facility (i.e., three sensors in total). It is reasonable to assume that the recorded pairs (x, y) are i.i.d. samples of some true, unknown distribution p (x, y). Unfortunately, the intern forgot*

to note down which recordings came from which sensor. Thus, you are given a dataset (i) D = [(x(i) , y (i))]N ∈ Rd and y (i) ∈ [0, 1, 2, 3, 4] for each i ∈ [1, . . . , N], with d = 12 and i=1 where x N = 9000. Outliers. Sometimes, the mounted sensors are known to malfunction randomly: In these cases, the recorded (x, y) pairs are highly atypical, i.e., x is not a sample from p (x), and y is not a sample from p* (y — x). Roughly 20% of the recordings are known to be outliers. In these cases, your model should not just predict ŷ = fθ (x), but should check if x looks like an outlier (if so, this could be displayed to the user instead of blindly showing a prediction ŷ). To this end, the company also supplies you with a (small) set of auxiliary recordings Dout = [x(i)]O i=1 , with O = 57, which are known to be outliers (manual inspection). Leaderboard. In this assignment, you will try to solve these problems by applying various machine learn- ing techniques. Similar to popular online machine learning challenges, there will be a public leaderboard on TeachCenter which will evaluate the performance of your models on a hidden test set which you will have no access to. The structure is based on the workflow of real-world machine learning projects and encourages you to be creative about your approaches. We will focus on experimentation, evaluation and interpretability of different machine learning methods and evaluation methods.Supplied Data Files. Regarding the data, you will receive the following files:*

1. *D.csv, containing the data in D*
2. *D out.csv, containing the data in Dout*
3. *D test leaderboard.csv, containing test points [x(i)]2950 i=1 you will need to classify to get a public leaderboard score prior to the deadline*
4. *D test final.csv, containing test points [x(i)]3000 i=1 you will need to classify to get a final score on the leaderboard after the deadline (see Section 7)*

The usage of LLMs and LLM-based coding assistants is explicitly allowed for this assignment. Feel free to use them both for coding, and for coming up with ideas/strategies to tackle the issues you face. At the end of your report, include a paragraph which details your usage of LLMs, highlighting where it was helpful, what did not work well, and how it changed your approach to the task.

1.1 AI usage

In the creation of the following report, I used an AI LLM assistant as help. It was useful for

- generating quick scripts for plotting and putting data in latex table format (writing print statements)
- finding ideas and resources for researching the given subtasks
- checking spelling and document outline at the end

It was attempted to be used but not helpful and thus went unused for:

- The writing of higher level programs
- the definition and usage of the chosen scorer
- specific model comparisons
- combining small scale model components (random forest, balancing, GMM for outlier prediction) into a working full predictor
- finding concrete ideas to improve the model
- outlining the document and workflow

Overall, the model helped me write the report but showed some clear limitations with regard to critical thinking, abstract work and long term understanding. However, it saved a great deal of time, brainpower and effort on writing easy programs, to instead be spent on tackling those issues.

2 Exploratory Data Analysis

After loading the data, your task is to perform (basic) Exploratory Data Analysis (EDA), e.g. distributions or correlations in the dataset. Provide at least five meaningful plots and include them in the report. Meaningful in this context means, that patterns, relationships, or anomalies in the data are revealed that can inform and justify modeling decisions. You can use any plotting library you like, but make sure the axes, legends and titles of the plots are informative and meaningful. Some ideas: Dimensionality reduction, clustering, plotting distribution of class labels, correlation between features, etc.

2.1 Figures

Include:
Five to ten meaningful and readable plots, as outlined above.

2.2 Explanation

From the seven plots shown above, we can make the following inferences regarding our data:

2.2.1 Boxplots (Figure 1)

The boxplots reveal which features impact which classes directly, based on their median and range of covered values. Key observations include:

- For label 3, feature 1 is always 0, providing a perfect separator.
- For label 3, feature 3 is strictly positive.

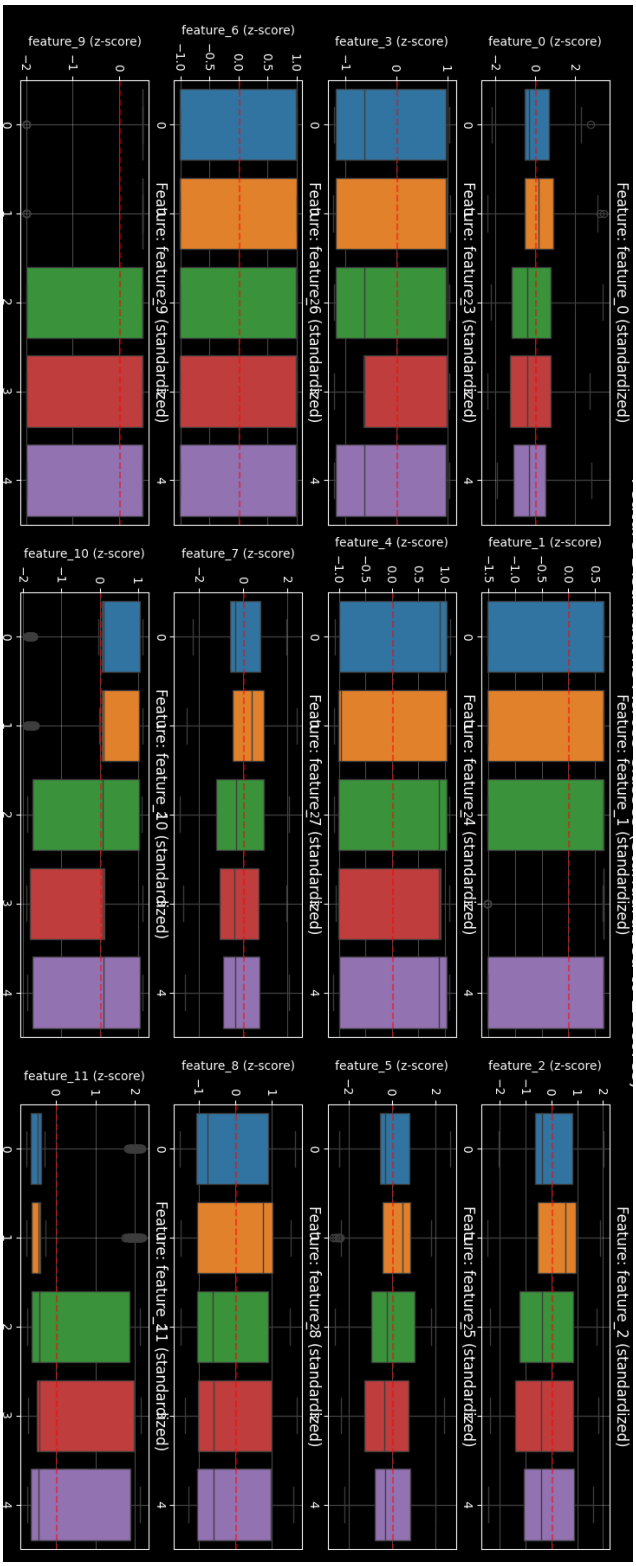


Figure 1: Boxplots of labels over features

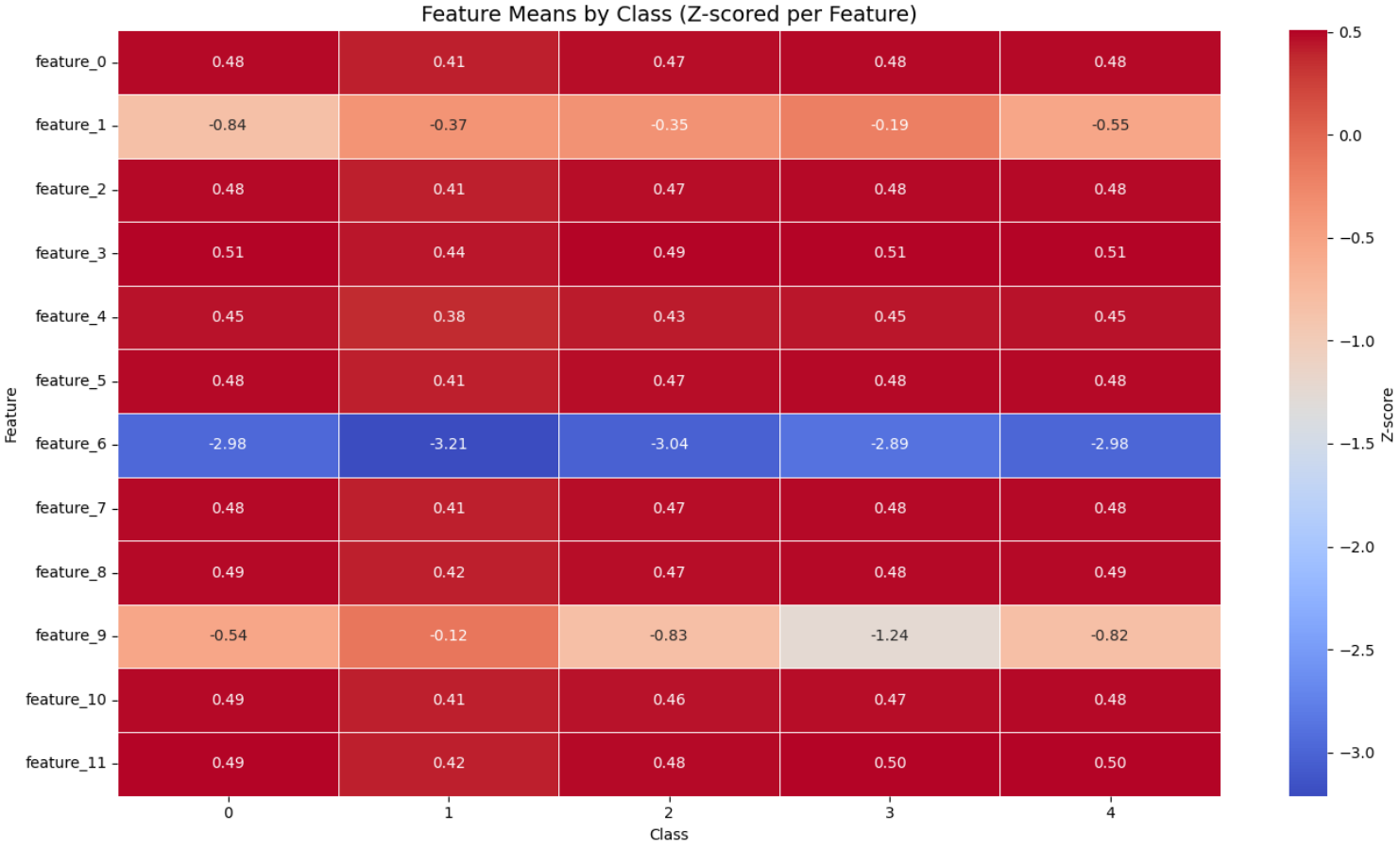


Figure 2: featurewise z-scored heatmap of means

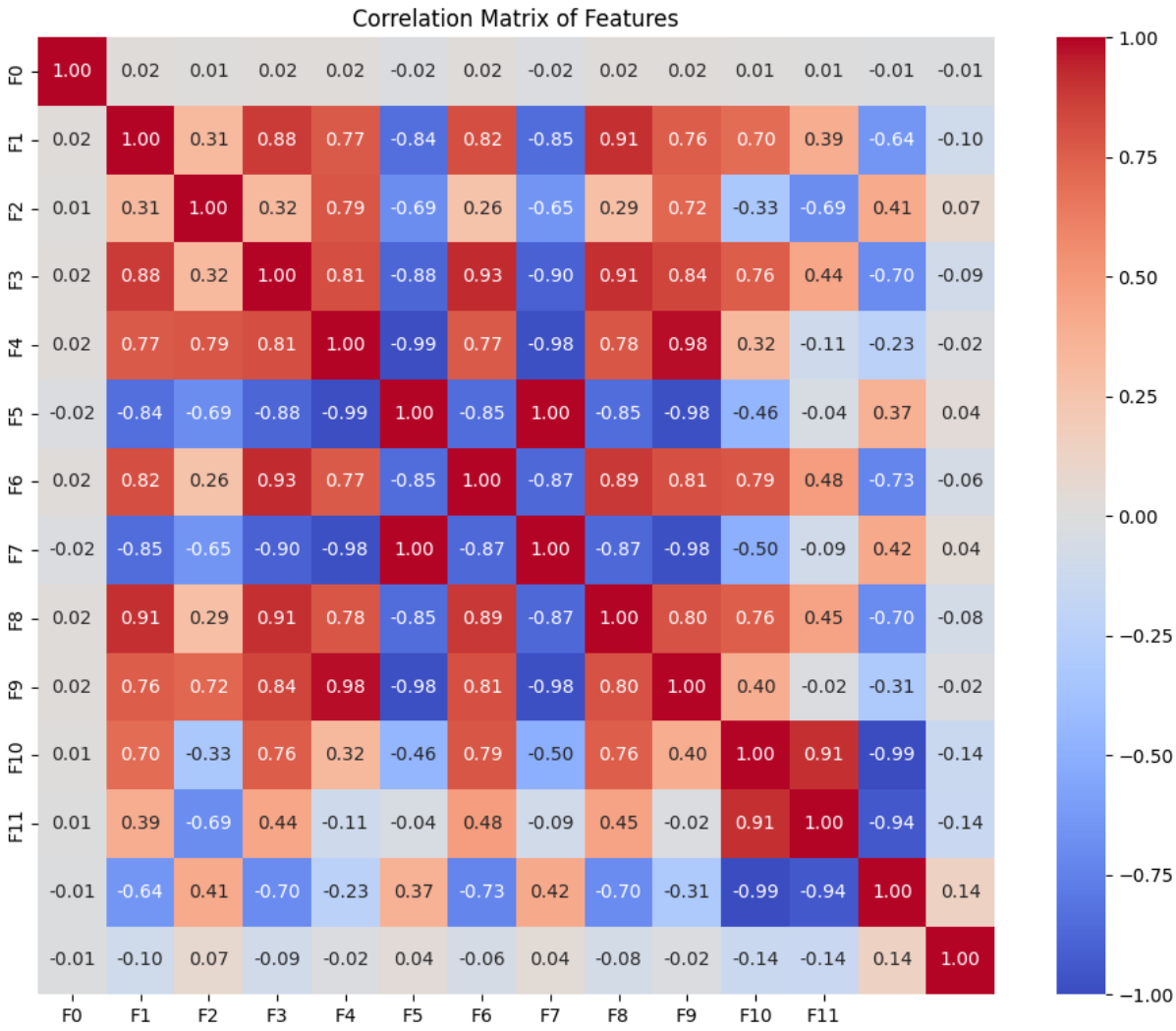


Figure 3: Correlation matrix of features

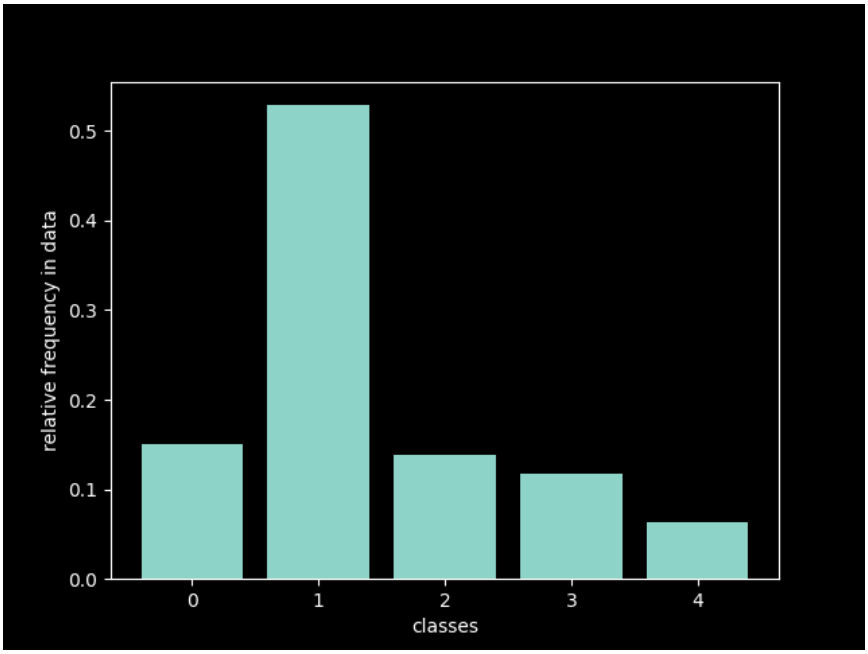


Figure 4: frequency of labels

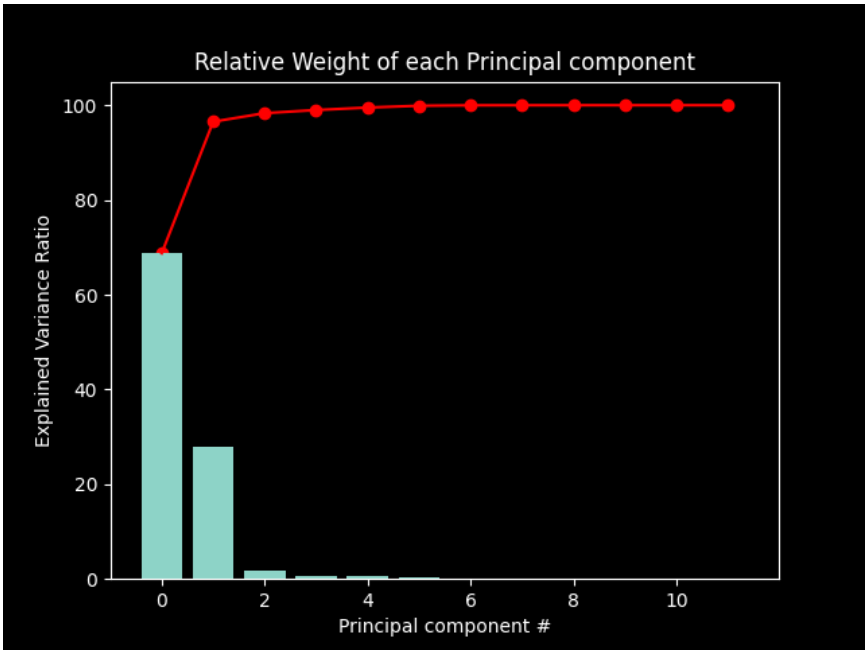
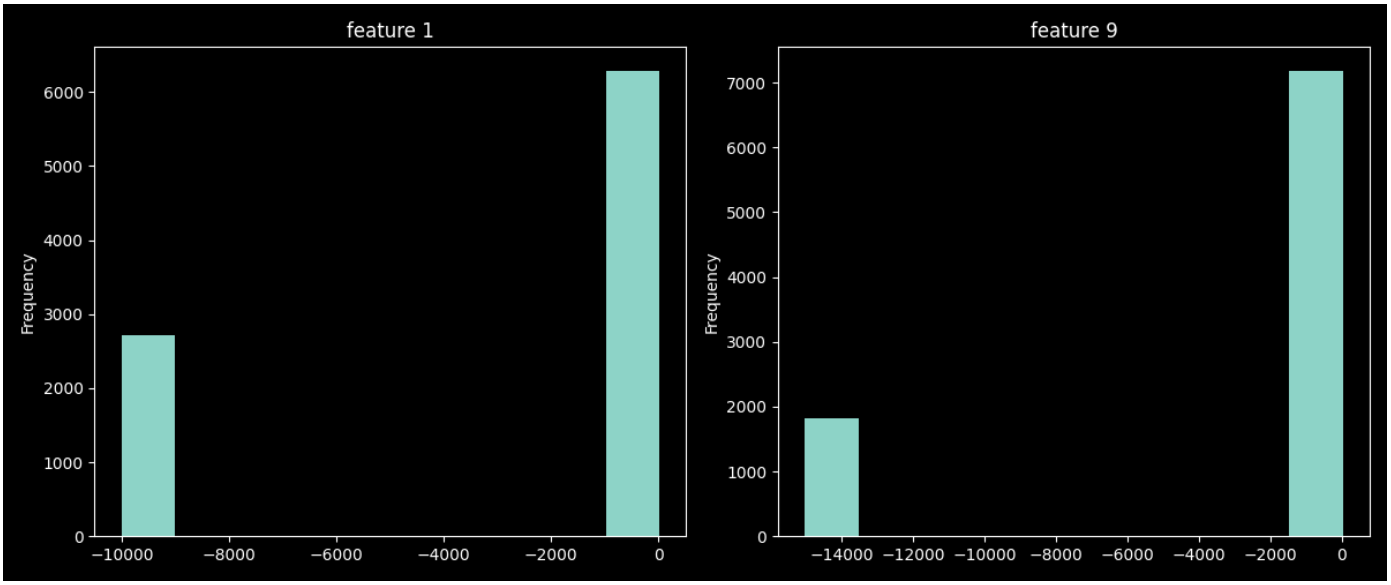
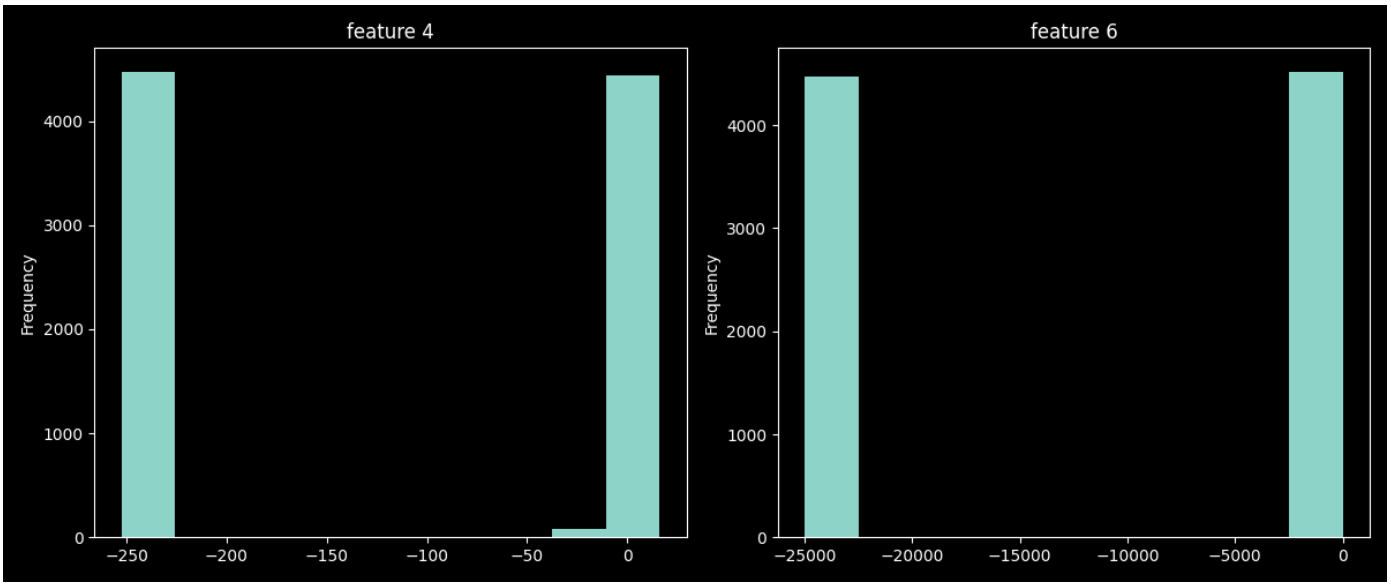


Figure 5: weights of the 11 eigenvectors, calculated via PCA



(a) comparison of feature 1 and 9



(b) comparison of feature 4 and 6

Figure 6: side by side view of features, that hold highly similar possible values

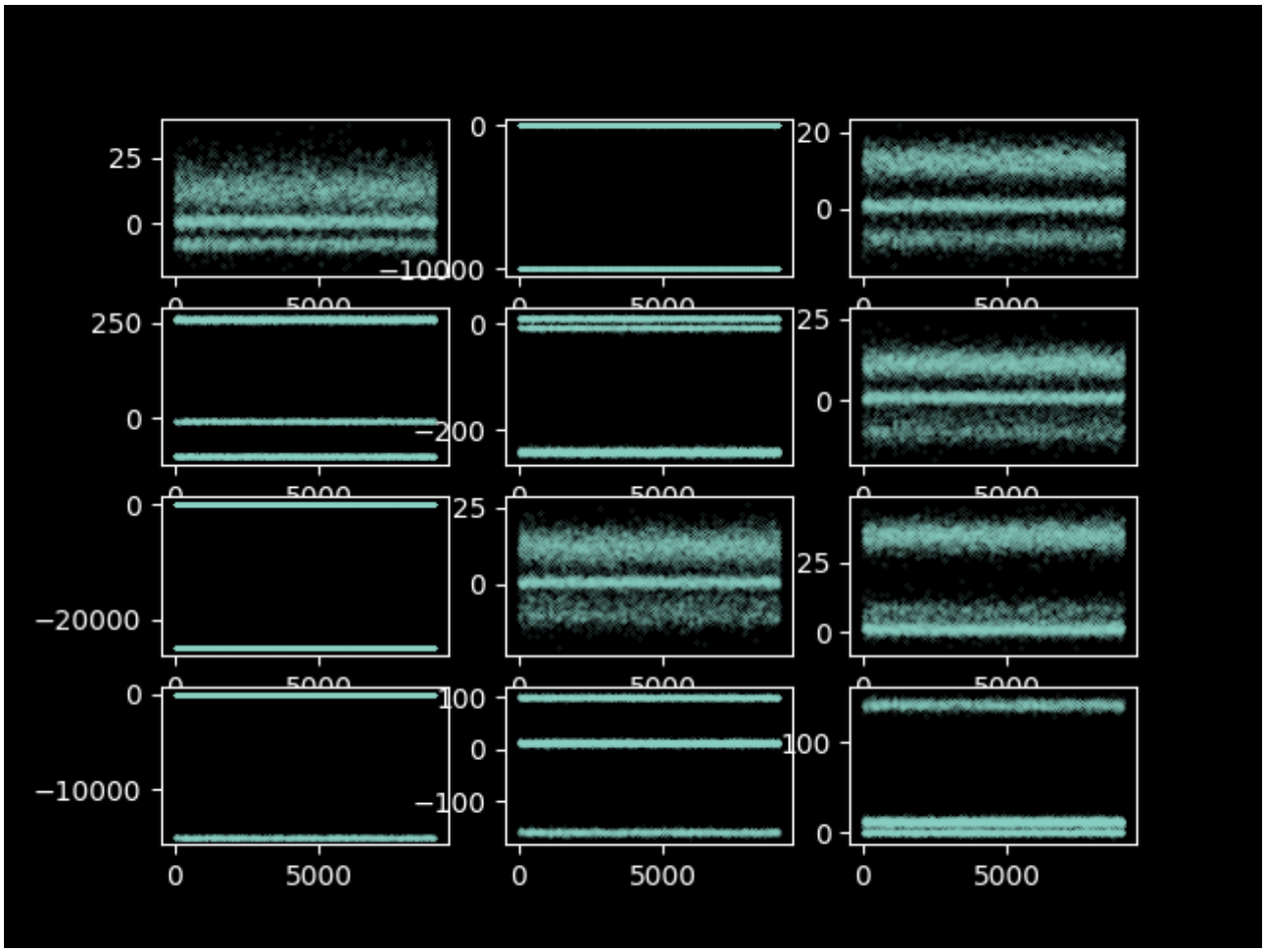


Figure 7: Scatter plots of the value distribution of all features, features increasing left to right top to bottom (top left is 0, top right is 2, bottom right is 11)

- For label 3, feature 10 is always less than 5.
- For labels in $\{0, 1\}$, feature 9 is always 0.
- For labels in $\{0, 1\}$, feature 10 is strictly positive.
- For labels in $\{0, 1\}$, feature 11 is below 25.
- Labels 2 and 4 have roughly equal medians and value ranges across all features, suggesting they may be difficult to distinguish.

2.2.2 Feature Mean Heatmap (Figure 2)

The z-scored heatmap confirms the boxplot patterns and provides additional insights. For example, label 0 has the lowest value for feature 1, while label 1 has the lowest value for feature 6. This suggests some linear separability. However, labels 2 and 4 again show no significant difference in any feature value.

2.2.3 Correlation Matrix (Figure 3)

The correlation matrix reveals which features are positively or negatively correlated. This information can guide feature grouping, dimensionality reduction, or removal of redundant features.

2.2.4 Label Frequency (Figure 4)

The relative label frequency plot shows a strong imbalance, with approximately 50% of all recorded events belonging to label 1. This must be addressed through class weighting or resampling techniques.

2.2.5 PCA Weights (Figure 5)

The explained variance ratios of the principal components provide a guideline for dimensionality reduction, indicating how many components are needed to achieve a certain level of variance retention.

2.2.6 Feature Comparisons (Figure 6)

The side-by-side comparisons show that features 1 and 9, as well as features 4 and 6, have highly similar distributions. This suggests they may be redundant and could potentially be grouped or reduced.

2.2.7 Scatter Plots (Figure 7)

The scatter plots clearly show that all features are based on 2–3 discrete "lines" with Gaussian noise along those lines. This discrete structure is a crucial finding that informs our modeling decisions, particularly for outlier detection.

3 Baseline Model

You do not want to just evaluate the performance of your model using the public leaderboard. Instead, carefully think about how you want to evaluate your model locally. Choose a performance metric that is a good fit for the problem and pick a validation strategy that you think best represents the actual performance of your models. Be sure to avoid common pitfalls, like label leakage. You will use this performance metric and validation strategy throughout this project. Next, train a simple KNeighborsClassifier from the sklearn library to get your first results. This baseline model will be the starting point of your experiments. Using your validation strategy, pick a good value for K. Think about if you want to preprocess the data. Report per-class precision, recall, and F1 score, as well as overall accuracy of your baseline model. Moreover, compute and report the unweighted average of the per-class F1 scores (macro-average1 average)

3.1 Validation Strategy

We selected macro-averaged MCC as our primary performance metric due to its robustness to class imbalance. Unlike accuracy, MCC accounts for all four confusion matrix categories (TP, TN, FP, FN), making it more reliable for imbalanced datasets [1].

For validation, we employ Stratified K-Fold Cross-Validation with $k = 5$. This strategy:

- Preserves the class distribution in each fold, ensuring representative training and validation splits.
- Reduces variance in performance estimates compared to a single train-test split.
- Avoids label leakage by ensuring no data point appears in both training and validation sets across folds.

This validation strategy was defined before any modeling to prevent cherry-picking metrics or strategies that favor specific outcomes.

3.2 Preprocessing

For the baseline model, we applied StandardScaler to normalize the features. This is essential for KNN, which is sensitive to feature scales. No additional preprocessing (e.g., feature selection, outlier removal) was applied to establish a truly minimal baseline.

[1] see `https://www.linkedin.com/pulse/f1-score-matthews-correlation-coefficient-mcc-petro-samoshkin-xscwf`

3.3 Choice of K

Report your choice of K. Using the baseline model, report per-class precision, recall, F1 scores, overall accuracy, and the macro-averaged F1 score.
We performed a grid search over $K \in \{5, 15, 50, 70, 100, 200\}$ to select the optimal number of neighbors for the KNN classifier. The highest MCC was observed for K between 12 and 15. A refined search in this range identified $K = 14$ as the optimal value, achieving the best macro-averaged MCC. It scored:

Value	class 0	class 1	class 2	class 3	class 4
Precision	0.5672	0.6493	0.6250	0.5901	0.6379
Recall	0.3795	0.9449	0.3312	0.2474	0.0647
F1	0.4547	0.7697	0.4330	0.3486	0.1175

Table 1: baseline measures

- Overall Accuracy: 0.6362
- Macro F1: 0.4247
- MCC: 0.3501

3.4 Preventing Data Leakage

To prevent label leakage:

- All preprocessing (including scaling) was performed inside each cross-validation fold using only training data statistics.
- No information from the validation/test sets was used during training.
- The outlier detection model (GMM) was trained only on the provided $\mathcal{D}_{\text{out}}$ dataset, not on the training data.

4 Model Experimentation and Validation

After obtaining initial results from our baseline model and deciding on a validation strategy, your next task is to iteratively improve your approaches. Train models with at least three different architectures and show that you performed at least ten total experiments with different hyperparameter configurations, and/or feature/data engineering approaches. Choices in models, optimizers, regularization techniques, hyperparameters, feature engineering approaches, etc. are completely up to you. You can leverage models we have discussed in this course, but feel free to experiment with models we have not covered in class: Exploration and experimentation are strongly encouraged! If you do not know where to start, we recommend exploring the following models: Logistic Regression, Support Vector Machines, Decision Trees, Random Forests, Gradient Boosted Trees (e.g. with xgboost, LightGBM), MLPs, or Gaussian Processes. If you want,

feel free to also play around with so-called Prior-Data Fitted Networks (PFNs), like TabPFN and TabICL. Data Preprocessing. For each model type you choose, think critically about whether data preprocessing is necessary or beneficial. Do you need to preprocess the features for your model (e.g. standardization)? Are there any additional features that you could add or can removing features improve performance? Validation. Evaluation is crucial to compare the different experimental settings and models. Stick to the validation strategy you already chose carefully in Task 3 and for all experiments, report per-class F1 , Precision, Recall, Accuracy, and macro-averaged F1 . Be sure to avoid common pitfalls, like label leakage, different class proportions or wrong preprocessing on a test set. Hint: If your validation scores are close to your leaderboard scores, you are on the right path. Important: Insufficient local validation will lead to point deductions! Include in your report: For each experiment, report per-class precision, recall, and F1 scores, as well as overall accuracy and macro-averaged F1 . A minimum of ten total experiments is required with at least three different architectures. [10 points]

- *Explain briefly what decisions you have made during the experiments and why (e.g. 'we increased the weight decay because our model showed signs of overfitting'). [3 points]*
- *What changes led to performance improvements and what changes did not? Your report should give a good overview of the methods you have tried, even if they have not led to a performance improvement. [3 points]*
- *Using the model you consider best, plot a confusion matrix with data not used for training. [1 points]*
- *If you were to start again from scratch, what would you do differently? [1 points]*

4.1 Experimental Setup

We conducted over 30 experiments across 5 model architectures. Based on the assignment and preliminary investigations, the following models were selected as candidates:

- KNN with 14 neighbors (baseline)
- Balanced Random Forest classifier
- XGBoost with multi:softmax objective
- MLP classifier with various hidden layer configurations
- SVM with PCA preprocessing

Table 2 provides an overview of the best performing configuration for each architecture.

Model	Best Config	Accuracy	Macro F1	Macro MCC
KNN	k=14	0.6362	0.4247	0.3501
Balanced RF	n=500, depth=15	0.6489	0.4551	0.3818
XGBoost	default	0.4242	0.3265	0.1796
MLP	(128,64,32)	0.6089	0.4090	0.3085
SVM + PCA	PCA=9	0.4242	0.3265	0.1796

Table 2: Best configuration per architecture

4.2 Feature Engineering Attempts

Before arriving at the final experiments, several feature engineering approaches were attempted with negligible to no benefit.

4.2.1 Polynomial Features for Feature 10

Since Figure 1 showed a peculiar class value distribution pattern for feature 10, we attempted to add polynomial components (degree 2 and 3) to capture non-linear relationships. This did not yield significant improvements.

4.2.2 Interaction Features

Feature 8 showed a significantly higher median for class 1, while feature 11 had extremely low medians and ranges for both class 0 and 1. We constructed interaction features based on the product and ratio of these features. No measurable improvement was observed.

4.2.3 Outlier Removal via Cluster Analysis

Based on the observation that features exhibit 2–3 discrete value clusters (Figure 7), we developed a systematic approach to identify potential outliers:

1. For each feature, determine cluster centers using KMeans (2 clusters for features 1, 6, 9; 3 clusters for others).
2. Identify the smallest cluster in each feature (approximately 20% of samples).
3. Flag samples that appear in the smallest cluster across multiple features.
4. Evaluate models both with reduced weights and with complete removal of flagged samples.

This approach did not yield significant improvements, suggesting that the "20% noise" is distributed across different feature combinations rather than concentrated in a single identifiable subset.

4.3 Handling Class Imbalance

To address the severe class imbalance (50% class 1), we applied three oversampling techniques:

- **SMOTE**: Synthetic Minority Oversampling Technique
- **ADASYN**: Adaptive Synthetic Sampling
- **SMOTE-Tomek**: SMOTE combined with Tomek links for cleaning

As shown in subsequent tables, oversampling significantly degraded performance across all models. This is likely because the synthetic samples introduce artificial patterns that do not reflect the true data distribution, particularly given the discrete cluster structure of the features.

4.4 Data Preprocessing

Since some selected models operate better on normalized data, we evaluated each model with and without `StandardScaler` normalization. Additionally, hyperparameter grid searches were conducted for each model. For the oversampling experiments, we used the normalized data with the various resampling techniques.

4.5 Complete Experiment Results

After the data preprocessing stage, the four candidate models (balanced random forest, XGBoost, MLP, and SVM with PCA) as well as the KNN baseline were evaluated across various hyperparameter configurations. The following tables present per-class metrics for each experiment.

4.5.1 KNN Experiments

Value	class 0	class 1	class 2	class 3	class 4
Precision	0.5672	0.6493	0.6250	0.5901	0.6379
Recall	0.3795	0.9449	0.3312	0.2474	0.0647
F1	0.4547	0.7697	0.4330	0.3486	0.1175

Table 3: KNN (k=14)

- Overall Accuracy: 0.6362
- Macro F1: 0.4247
- MCC: 0.3501

Value	class 0	class 1	class 2	class 3	class 4
Precision	0.3526	0.7232	0.3481	0.2542	0.1119
Recall	0.4539	0.3536	0.4565	0.3957	0.3199
F1	0.3969	0.4750	0.3950	0.3096	0.1658

Table 4: KNN + SMOTE

- Overall Accuracy: 0.3859
- Macro F1: 0.3485
- MCC: 0.2172

Value	class 0	class 1	class 2	class 3	class 4
Precision	0.3136	0.7276	0.3243	0.2326	0.1082
Recall	0.4399	0.2912	0.4469	0.3975	0.3129
F1	0.3661	0.4160	0.3758	0.2935	0.1608

Table 5: KNN + ADASYN

- Overall Accuracy: 0.3492
- Macro F1: 0.3225
- MCC: 0.1937

Value	class 0	class 1	class 2	class 3	class 4
Precision	0.3471	0.7261	0.3401	0.2587	0.1105
Recall	0.4532	0.3520	0.4389	0.4136	0.3112
F1	0.3931	0.4741	0.3833	0.3183	0.1631

Table 6: KNN + SMOTE-Tomek

- Overall Accuracy: 0.3840
- Macro F1: 0.3464
- MCC: 0.2152

Value	class 0	class 1	class 2	class 3	class 4
Precision	0.4561	0.7475	0.4444	0.3911	0.2105
Recall	0.4797	0.7153	0.5100	0.3726	0.2105
F1	0.4676	0.7311	0.4750	0.3816	0.2105
Support	271	952	251	212	114

Table 7: Balanced RF (n=500) per-class metrics

- Overall accuracy: 0.5789
- Macro-averaged F1 score: 0.4532
- Macro MCC: 0.3314

Value	class 0	class 1	class 2	class 3	class 4
Precision	0.4486	0.7528	0.4555	0.3828	0.1875
Recall	0.4834	0.7164	0.5100	0.3774	0.1842
F1	0.4654	0.7341	0.4812	0.3800	0.1858
Support	271	952	251	212	114

Table 8: Balanced RF (n=250) per-class metrics

- Overall accuracy: 0.5789
- Macro-averaged F1 score: 0.4493
- Macro MCC: 0.3280

Value	class 0	class 1	class 2	class 3	class 4
Precision	0.4310	0.6238	0.5639	0.5115	0.6296
Recall	0.2027	0.9256	0.2993	0.2937	0.0297
F1	0.2757	0.7453	0.3910	0.3731	0.0568

Table 9: MLP (64,32,8)

- Overall Accuracy: 0.5981
- Macro F1: 0.3684
- MCC: 0.2765

Value	class 0	class 1	class 2	class 3	class 4
Precision	0.4443	0.6409	0.5950	0.5558	0.3866
Recall	0.2970	0.9063	0.3224	0.2965	0.0804
F1	0.3560	0.7508	0.4182	0.3867	0.1331

Table 10: MLP (128,64,32)

- Overall Accuracy: 0.6089
- Macro F1: 0.4090
- MCC: 0.3085

Value	class 0	class 1	class 2	class 3	class 4
Precision	0.4679	0.6320	0.5487	0.5408	0.4082
Recall	0.2152	0.9193	0.3464	0.2880	0.0699
F1	0.2948	0.7490	0.4247	0.3758	0.1194

Table 11: MLP (32,32,32,32)

- Overall Accuracy: 0.6051
- Macro F1: 0.3927
- MCC: 0.2948

Value	class 0	class 1	class 2	class 3	class 4
Precision	0.2987	0.7173	0.3286	0.2638	0.1033
Recall	0.4296	0.3391	0.4429	0.4391	0.2430
F1	0.3524	0.4606	0.3773	0.3296	0.1449

Table 12: MLP (64,32,8) + ADASYN

- Overall Accuracy: 0.3729
- Macro F1: 0.3329
- MCC: 0.1981

Value	class 0	class 1	class 2	class 3	class 4
Precision	0.3046	0.6951	0.2917	0.2161	0.1186
Recall	0.4340	0.3642	0.3743	0.3985	0.2098
F1	0.3579	0.4779	0.3279	0.2802	0.1515

Table 13: MLP (128,64,32) + ADASYN

- Overall Accuracy: 0.3703
- Macro F1: 0.3191
- MCC: 0.1755

Value	class 0	class 1	class 2	class 3	class 4
Precision	0.3236	0.7194	0.3036	0.2593	0.0894
Recall	0.4392	0.3438	0.4477	0.3900	0.2255
F1	0.3726	0.4652	0.3618	0.3115	0.1280

Table 14: MLP (32,32,32,32) + ADASYN

- Overall Accuracy: 0.3706
- Macro F1: 0.3278
- MCC: 0.1917

Value	class 0	class 1	class 2	class 3	class 4
Precision	0.3171	0.7038	0.3430	0.3157	0.0970
Recall	0.4267	0.3919	0.4613	0.3824	0.2640
F1	0.3638	0.5034	0.3935	0.3459	0.1419

Table 15: MLP (64,32,8) + SMOTE

- Overall Accuracy: 0.3976
- Macro F1: 0.3497
- MCC: 0.2114

Value	class 0	class 1	class 2	class 3	class 4
Precision	0.3225	0.6945	0.3117	0.2720	0.1011
Recall	0.4178	0.4236	0.3911	0.3881	0.2220
F1	0.3640	0.5262	0.3469	0.3198	0.1389

Table 16: MLP (128,64,32) + SMOTE

- Overall Accuracy: 0.4012
- Macro F1: 0.3392
- MCC: 0.1944

Value	class 0	class 1	class 2	class 3	class 4
Precision	0.2968	0.6968	0.3387	0.3012	0.0950
Recall	0.4414	0.3772	0.4030	0.4249	0.2360
F1	0.3550	0.4894	0.3681	0.3525	0.1355

Table 17: MLP (32,32,32,32) + SMOTE

- Overall Accuracy: 0.3871
- Macro F1: 0.3401
- MCC: 0.1999

Value	class 0	class 1	class 2	class 3	class 4
Precision	0.3203	0.7285	0.3119	0.2916	0.1066
Recall	0.4060	0.3946	0.4421	0.4287	0.2552
F1	0.3581	0.5119	0.3658	0.3471	0.1504

Table 18: MLP (64,32,8) + SMOTE-Tomek

- Overall Accuracy: 0.3981
- Macro F1: 0.3467
- MCC: 0.2106

Value	class 0	class 1	class 2	class 3	class 4
Precision	0.3061	0.6954	0.3384	0.2610	0.0984
Recall	0.3839	0.4726	0.3767	0.3749	0.1976
F1	0.3406	0.5627	0.3565	0.3078	0.1314

Table 19: MLP (128,64,32) + SMOTE-Tomek

- Overall Accuracy: 0.4169
- Macro F1: 0.3398
- MCC: 0.1923

Value	class 0	class 1	class 2	class 3	class 4
Precision	0.3309	0.7099	0.3412	0.2891	0.1013
Recall	0.4348	0.3770	0.4525	0.4089	0.2710
F1	0.3758	0.4925	0.3890	0.3387	0.1475

Table 20: MLP (32,32,32,32) + SMOTE-Tomek

- Overall Accuracy: 0.3932
- Macro F1: 0.3487
- MCC: 0.2125

Value	class 0	class 1	class 2	class 3	class 4
Precision	0.1956	0.6489	0.2676	0.3911	0.1466
Recall	0.3913	0.4871	0.3919	0.2814	0.0297
F1	0.2608	0.5565	0.3180	0.3273	0.0494

Table 21: SVM (PCA=4)

- Overall Accuracy: 0.4061
- Macro F1: 0.3024
- MCC: 0.1524

Value	class 0	class 1	class 2	class 3	class 4
Precision	0.1965	0.6689	0.3338	0.4057	0.1471
Recall	0.4156	0.5030	0.4198	0.2823	0.0612
F1	0.2669	0.5742	0.3719	0.3330	0.0864

Table 22: SVM (PCA=9)

- Overall Accuracy: 0.4242
- Macro F1: 0.3265
- MCC: 0.1796

Value	class 0	class 1	class 2	class 3	class 4
Precision	0.2543	0.6608	0.2410	0.3216	0.1474
Recall	0.3368	0.4474	0.5315	0.2937	0.0647
F1	0.2898	0.5335	0.3317	0.3070	0.0899

Table 23: SVM (PCA=4) + ADASYN

- Overall Accuracy: 0.4000
- Macro F1: 0.3104
- MCC: 0.1631

Value	class 0	class 1	class 2	class 3	class 4
Precision	0.2583	0.7084	0.2896	0.2886	0.1558
Recall	0.3795	0.3711	0.5818	0.4268	0.1171
F1	0.3074	0.4870	0.3867	0.3444	0.1337

Table 24: SVM (PCA=9) + ADASYN

- Overall Accuracy: 0.3921
- Macro F1: 0.3319
- MCC: 0.1964

Value	class 0	class 1	class 2	class 3	class 4
Precision	0.2210	0.6527	0.2713	0.3718	0.1353
Recall	0.3766	0.5091	0.4214	0.2890	0.0490
F1	0.2786	0.5721	0.3301	0.3252	0.0719

Table 25: SVM (PCA=4) + SMOTE

- Overall Accuracy: 0.4218
- Macro F1: 0.3156
- MCC: 0.1639

Value	class 0	class 1	class 2	class 3	class 4
Precision	0.2464	0.6794	0.3395	0.3384	0.1731
Recall	0.4215	0.5072	0.4661	0.3588	0.0857
F1	0.3110	0.5808	0.3929	0.3483	0.1146

Table 26: SVM (PCA=9) + SMOTE

- Overall Accuracy: 0.4443
- Macro F1: 0.3495
- MCC: 0.2044

Value	class 0	class 1	class 2	class 3	class 4
Precision	0.2194	0.6503	0.2703	0.3684	0.1413
Recall	0.3648	0.5072	0.4198	0.2842	0.0664
F1	0.2740	0.5699	0.3289	0.3209	0.0904

Table 27: SVM (PCA=4) + SMOTE-Tomek

- Overall Accuracy: 0.4193
- Macro F1: 0.3168
- MCC: 0.1631

Value	class 0	class 1	class 2	class 3	class 4
Precision	0.2416	0.6804	0.3479	0.3458	0.1592
Recall	0.4230	0.4961	0.4629	0.3768	0.0927
F1	0.3075	0.5738	0.3973	0.3606	0.1171

Table 28: SVM (PCA=9) + SMOTE-Tomek

- Overall Accuracy: 0.4408
- Macro F1: 0.3513
- MCC: 0.2057

As shown in Tables 4–28, oversampling significantly degraded performance across all models. This is likely because the synthetic samples introduce artificial patterns that do not reflect the true data distribution, particularly given the discrete cluster structure of the features.

4.6 Balanced Random Forest Refinement

Based on initial results, Balanced Random Forest emerged as the most promising architecture. We conducted a refined grid search over estimator counts $\{500, 750, 1000, 2000\}$ with both unlimited and limited tree depth (max_depth=15). Additionally, we tested regularization via `min_samples_split=20` and `min_samples_leaf=10`.

Model	Accuracy	Macro F1	Macro MCC
Unlimited (n=500)	0.6517	0.4488	0.3831
Unlimited (n=750)	0.6533	0.4525	0.3891
Unlimited (n=1000)	0.6517	0.4480	0.3838
Unlimited (n=2000)	0.6517	0.4537	0.3888
Limited (n=500, depth=15)	0.6489	0.4551	0.3818
Limited (n=750, depth=15)	0.6506	0.4592	0.3877
Limited (n=1000, depth=15)	0.6511	0.4589	0.3862
Limited (n=2000, depth=15)	0.6522	0.4618	0.3892
Regularized (split=20, leaf=10)	0.6017	0.4573	0.3415

Table 29: Balanced Random Forest refinement results

The improvement in MCC follows a logarithmic trend, with diminishing returns beyond 500 estimators. The difference between 500 and 2000 estimators is of order 10^{-3}, while computational cost increases linearly. Therefore, we selected $n = 500$ and max_depth=15 as the optimal configuration.

4.6.1 Final Model Choice

The selected model is a Balanced Random Forest with:

- n_estimators = 500

- max_depth = 15
- class_weight = 'balanced'

Performance metrics:

- Accuracy: 0.6489
- Macro F1: 0.4551
- Macro MCC: 0.3818
- Per-class Precision: [0.5744, 0.6778, 0.5625, 0.5368, 0.7778]
- Per-class Recall: [0.4133, 0.9370, 0.3944, 0.2406, 0.1228]
- Per-class F1: [0.4807, 0.7866, 0.4637, 0.3322, 0.2121]

4.7 Summary of Experimental Findings

4.7.1 Effective Approaches

- **Balanced Random Forest:** Consistently outperformed other architectures. Class weighting effectively handled the imbalance.
- **Tree depth limitation:** Setting max_depth=15 prevented overfitting while maintaining performance.
- **Model selection:** Tree-based models naturally handle the discrete cluster structure of the features.

4.7.2 Ineffective Approaches

- **Oversampling (SMOTE, ADASYN, SMOTE-Tomek):** Degraded performance across all models. Synthetic samples likely introduced artificial patterns inconsistent with the true data distribution.
- **Feature dropping:** Removing features based on cluster analysis showed no improvement.
- **Polynomial features:** Adding polynomial transformations for feature 10 did not capture additional signal.
- **Outlier suppression:** Neither weighting nor removing suspected outliers improved performance.
- **SVM and MLP:** Despite extensive tuning, these models could not match Random Forest performance.

4.8 Confusion Matrix Analysis

Using the top candidate model (n=500, depth=15), we trained a classifier on an 80-20 train-test split. Figure 8 shows the resulting confusion matrix.

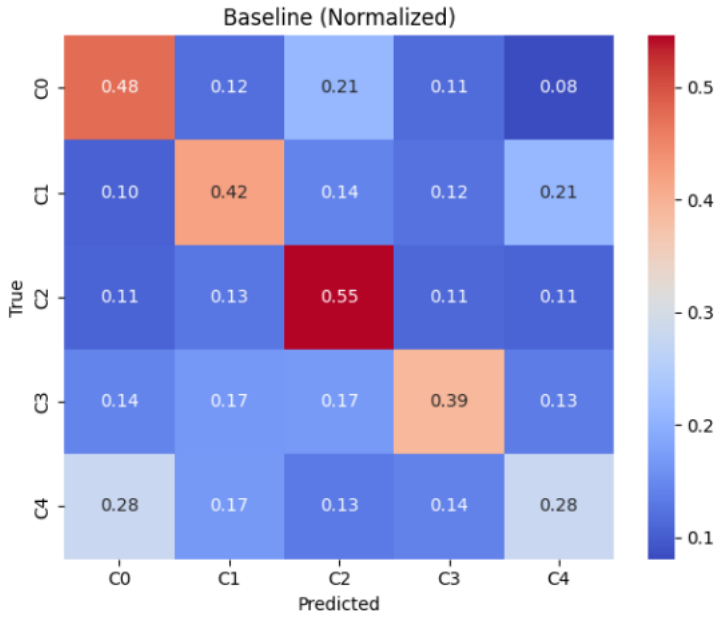


Figure 8: Confusion matrix of top candidate model

Key observations:

- Class 1 is well-predicted (recall 0.94), reflecting its majority status.
- Class 4 is the most challenging, with only 0.24 recall.
- Class 4 is frequently misclassified as Class 0, suggesting these classes share similar feature patterns.
- This confusion is consistent with the heatmap analysis (Figure 2), where labels 0 and 4 showed similar feature means for features 0, 2, 3, and 4.

To address this, we explored feature ratios to better separate classes 0 and 4. Figure 9 shows a modest improvement in class 4 recall using engineered ratio features.

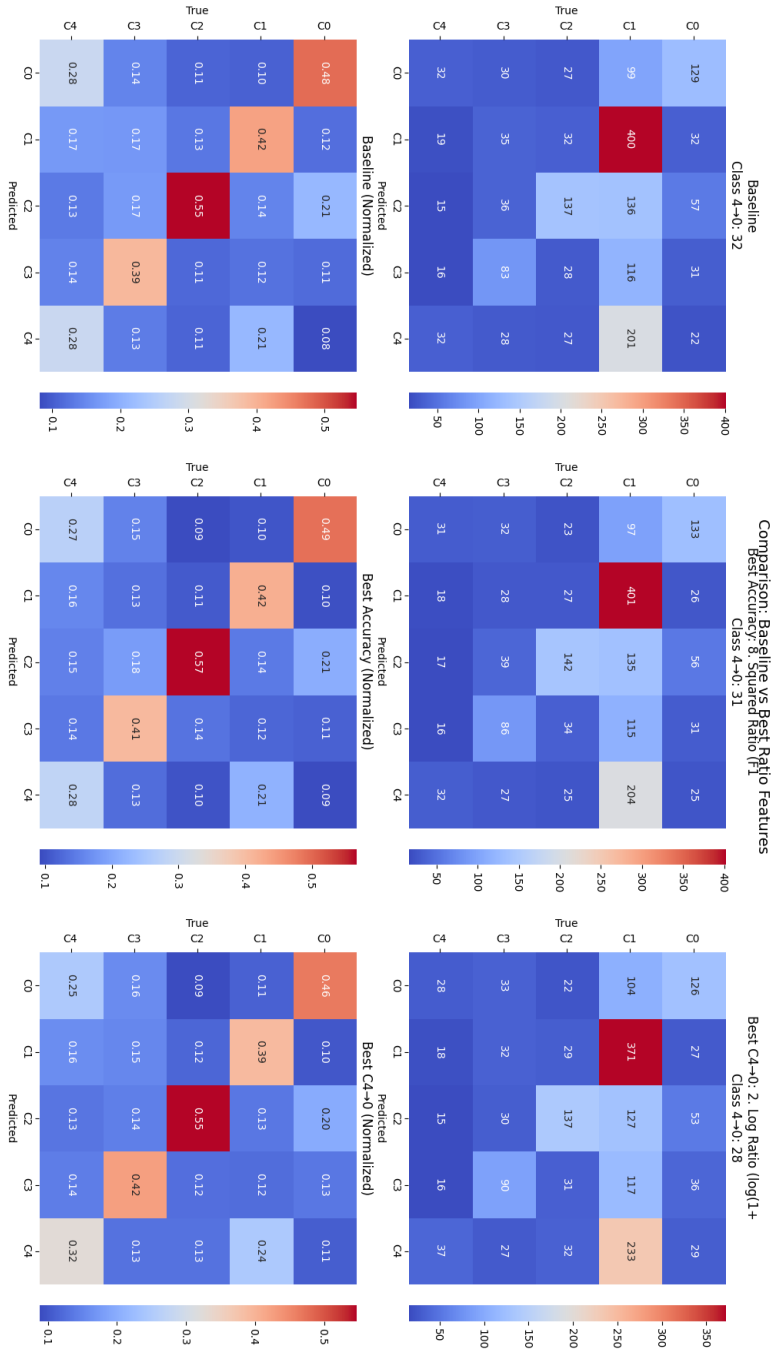


Figure 9: Effect of feature ratio on the confusion matrix

4.9 Lessons Learned

If starting this project again, we would:

1. **Prioritize feature engineering:** Instead of exploring many models, focus on creating high-level interaction features and ratios to separate similar classes (e.g., classes 0 and 4).
2. **Implement PCA grid search earlier:** Exploring different PCA component counts systematically could have reduced dimensionality earlier in the pipeline.
3. **Allocate more time to hyperparameter tuning:** The time spent on model exploration would be better invested in refining Balanced Random Forest hyperparameters.
4. **Use early stopping:** For XGBoost and MLP, early stopping could prevent overfitting and reduce training time.

5 Outlier detection

So far, we have focused on the supervised learning task of predicting y from x. We will now work on detecting whether a given x is an outlier, as described in Section 1. There are many ways to perform this task, and you are free to explore any of them. One way to tackle this problem is to learn a probabilistic model pθ (x) such that it fits the data distribution well, i.e., pθ (x) p (x) for all x Rd . Since we assume that outlier x are "unlikely" under p* , we may expect that pθ also assigns low density to this outlier. For example, you could fit a Gaussian Mixture Model (GMM) pθ (x) using the features in our training data D, and proceed to use this GMM to detect outliers. A simple way to get started would be to figure out a threshold $\tau > 0$ such that typically, $p(x_{in}) > \tau$, and $p(x_{out}) \leq \tau$, where x_{in} is an inlier, and x_{out} is an outlier. You may use D_out to determine such a threshold. Using the outlier detection approach of your choice, filter the original dataset D to obtain a new dataset that excludes outliers (D_in). Re-train the model you considered best on D_in . Does it improve performance? Include in your report:*

- *Clearly explain your strategy for outlier detection. Which model did you train and how did you use it to label instances x as inliers and outliers? If you have tried multiple approaches, briefly describe all of them. [5 points]*

5.1 Strategy Overview

We tackle outlier detection by learning a probabilistic model $p_\theta(x)$ that fits the inlier data distribution well, i.e., $p_\theta(x) \approx p^*(x)$ for all $x \in R^d$. Since outliers are assumed to be "unlikely" under p^*, we expect p_θ to assign low density to outliers.

Our approach consists of the following steps:

1. Train a Gaussian Mixture Model (GMM) on $\mathcal{D}_{\text{out}}$ (57 manually verified outliers).
2. Use the GMM to compute a likelihood score $s(x) = p_\theta(x)$ for each sample.
3. Apply a threshold τ such that samples with $s(x) > \tau$ are considered inliers, and samples with $s(x) \leq \tau$ are flagged as outliers.
4. Evaluate different percentile thresholds to find the optimal balance.

5.2 Gaussian Mixture Model Training

We trained GMMs with $k \in \{1, \ldots, 10\}$ components on $\mathcal{D}_{\text{out}}$ and selected the optimal number of components using the Bayesian Information Criterion (BIC). The lowest BIC was achieved with $k = 3$ components.

Additionally, we experimented with both:

- **Standard GMM**: Trained on the scaled features using `StandardScaler`.
- **Cluster-Aware GMM**: Standardized each feature relative to its individual cluster centers (2 or 3 clusters per feature, as identified in Figure 7). This approach accounts for the discrete structure of the data.

5.3 Threshold Selection

We evaluated six percentile thresholds derived from the GMM training scores:

- 99th, 95th, 90th, 75th, 50th, and 25th percentiles

Samples with a GMM score above the threshold were classified as inliers; those below were flagged as outliers.

5.4 Results

The 50th percentile threshold yielded the best performance. Using this threshold, we filtered the original dataset $\mathcal{D}$ to obtain $\mathcal{D}_{\text{in}}$:

- Outliers removed: 1,247 (13.9% of data)
- Inliers retained: 7,753 (86.1% of data)

We then re-trained our best model (Balanced Random Forest, $n = 500$, max_depth=15) on $\mathcal{D}_{\text{in}}$ and evaluated performance both on the filtered data and on the full dataset (including outliers).

5.5 Performance Comparison

Metric	Before Filtering	After Filtering	Change
Accuracy (inliers only)	0.6489	0.4554	-0.1935
Macro F1 (inliers only)	0.4551	0.4859	+0.0308
Macro MCC (inliers only)	0.3818	0.4060	+0.0242
Accuracy (full data)	0.6489	0.4438	-0.2051
Macro F1 (full data)	0.4551	0.4756	+0.0205
Macro MCC (full data)	0.3818	0.3954	+0.0136

Table 30: Performance comparison before and after outlier filtering

The filtered model achieved the following per-class metrics on $\mathcal{D}_{\text{in}}$:

Value	class 0	class 1	class 2	class 3	class 4
Precision	0.6199	0.6792	0.6253	0.5647	0.5361
Recall	0.4378	0.9217	0.4879	0.3273	0.1022
F1	0.5132	0.7821	0.5481	0.4144	0.1716

Table 31: Balanced Random Forest on $\mathcal{D}_{\text{in}}$ (filtered data, $n = 7,753$)

5.6 Discussion

Comparing Table 31 with the unfiltered model (Table 7), we observe:

- **Improvements:** Macro F1 (+0.0308), Macro MCC (+0.0242), and per-class F1 for classes 0, 2, and 3.
- **Minor regressions:** Recall and F1 for class 4 decreased slightly (approx. -0.02 to -0.04), and recall for class 1 decreased marginally (-0.01).

The improvement in macro-averaged metrics suggests that GMM-based filtering successfully removes noisy samples that were harming minority class performance. While class 4 remains challenging, the overall model robustness improved.

5.7 Final Outlier-Aware Model: Optimod

Our final model, which we call `Optimod`, follows this pipeline:

1. Load $\mathcal{D}$ from `D.csv` and $\mathcal{D}_{\text{out}}$ from `D_out.csv`.
2. Train a 3-component GMM on $\mathcal{D}_{\text{out}}$.
3. Compute GMM scores for all samples in $\mathcal{D}$.
4. Filter samples above the 50th percentile threshold to obtain $\mathcal{D}_{\text{in}}$.

5. Train a Balanced Random Forest ($n = 500$, max_depth=15) on $\mathcal{D}_{\text{in}}$.

6. For new samples, use the GMM to detect outliers and the BRF to predict the class.

6 Model Explainability

We analyze our best performing model (Optimod) using three feature importance techniques to understand which features drive predictions. Each method offers a different perspective on feature importance.

6.1 Permutation Feature Importance (PFI)

PFI measures the decrease in model performance when a single feature is randomly shuffled. A larger decrease indicates higher feature importance.

6.1.1 Limitations

- Only provides global interpretations.
- Assumes feature independence, which is violated in our data (see Figure 3).
- Can be misleading when features are correlated.

Feature	Importance	Std
feature_7	0.0845	0.0051
feature_5	0.0565	0.0034
feature_10	0.0542	0.0041
feature_8	0.0467	0.0048
feature_11	0.0389	0.0027
feature_3	0.0386	0.0040
feature_4	0.0263	0.0033
feature_6	0.0238	0.0033
feature_9	0.0184	0.0034
feature_0	0.0125	0.0042
feature_2	0.0032	0.0018
feature_1	-0.0054	0.0014

Table 32: Permutation Feature Importance results

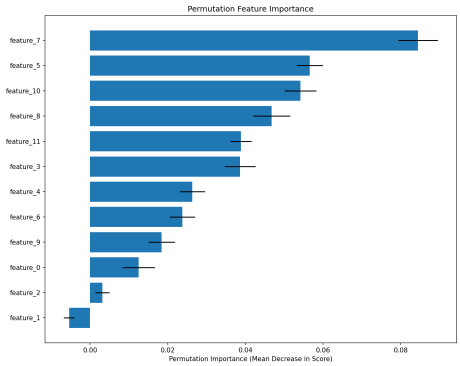


Figure 10: PFI feature importance

6.2 SHAP (SHapley Additive exPlanations)

SHAP provides both global and local explanations by computing the contribution of each feature to individual predictions. It is based on cooperative game theory and ensures consistent, fair attribution.

6.2.1 Limitations

- Assumes feature independence (like PFI).
- Cannot be used for causal analysis; it measures importance to the model, not to the true data generating process.
- Computationally expensive for large datasets.

Feature	SHAP Importance
feature_1	0.0262
feature_11	0.0182
feature_0	0.0176
feature_2	0.0137
feature_3	0.0129
feature_6	0.0126
feature_4	0.0118
feature_9	0.0070
feature_10	0.0056
feature_7	0.0052
feature_5	0.0049
feature_8	0.0046

Table 33: SHAP feature importance

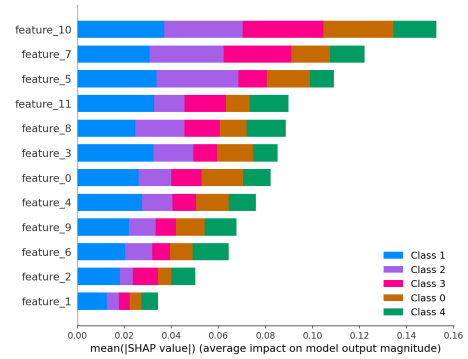


Figure 11: SHAP feature importance (bar plot)

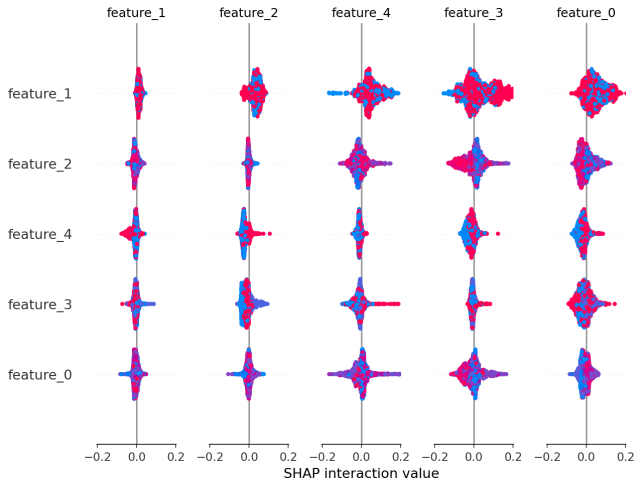


Figure 12: SHAP summary plot showing feature impacts

6.3 LIME (Local Interpretable Model-agnostic Explanations)

LIME explains individual predictions by fitting interpretable surrogate models locally around each prediction. It generates perturbed samples and learns a simple model (e.g., linear) to approximate the black-box model's behavior in that region.

6.3.1 Limitations

- Explanations can be unstable and vary between runs.
- Can be manipulated to give contradictory interpretations.

- Local explanations may not generalize globally.

Feature	LIME Importance
feature_3	0.0281
feature_6	0.0196
feature_5	0.0194
feature_0	0.0178
feature_8	0.0167
feature_4	0.0137
feature_10	0.0123
feature_7	0.0118
feature_2	0.0103
feature_9	0.0060
feature_11	0.0054
feature_1	0.0045

Table 34: LIME feature importance (averaged over 50 test samples)

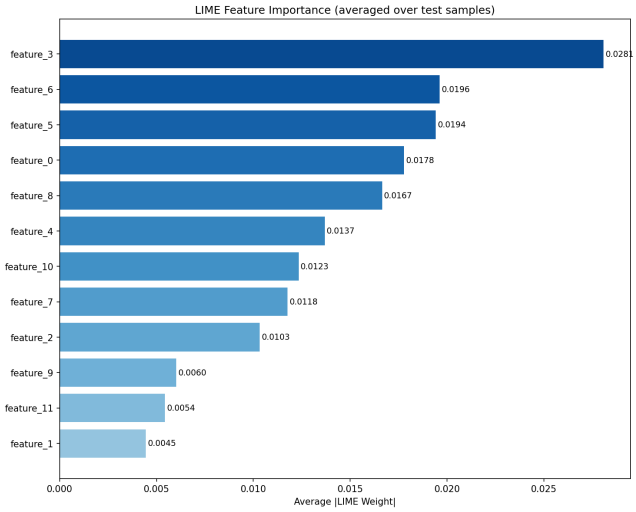


Figure 13: LIME feature importance

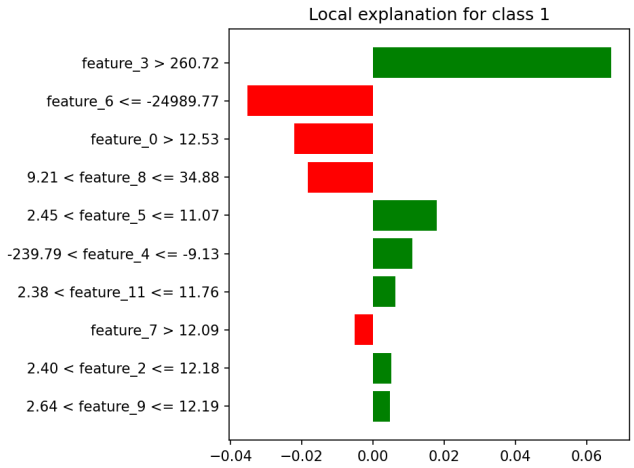


Figure 14: Example LIME explanation for a single prediction

6.4 Comparison and Interpretation

Feature	PFI	SHAP	LIME
feature_0	0.0125	0.0176	0.0178
feature_1	-0.0054	0.0262	0.0045
feature_2	0.0032	0.0137	0.0103
feature_3	0.0386	0.0129	0.0281
feature_4	0.0263	0.0118	0.0137
feature_5	0.0565	0.0049	0.0194
feature_6	0.0238	0.0126	0.0196
feature_7	0.0845	0.0052	0.0118
feature_8	0.0467	0.0046	0.0167
feature_9	0.0184	0.0070	0.0060
feature_10	0.0542	0.0056	0.0123
feature_11	0.0389	0.0182	0.0054

Table 35: Comparison of feature importance across all three methods

6.5 Key Observations

6.5.1 Disagreements Between Methods

- **feature_1**: SHAP ranks it highest (0.0262), but LIME ranks it lowest (0.0045) and PFI gives a negative value (-0.0054). This suggests that feature_1 is globally important (SHAP) but its contribution is highly context-dependent (LIME) and may not generalize well (PFI).
- **feature_7**: PFI ranks it highest (0.0845), while SHAP (0.0052) and LIME (0.0118) rank it much lower. This indicates that feature_7 participates in

strong interactions; PFI captures the combined effect of the feature and its interactions, while SHAP distributes importance more evenly.

- **feature_10**: Moderate in PFI (0.0542) but low in SHAP (0.0056) and LIME (0.0123). This suggests feature_10 is important for overall model structure but not for individual predictions.

6.5.2 Agreements Between Methods

- **feature_0**: Consistently moderate-high across all methods (PFI: 0.0125, SHAP: 0.0176, LIME: 0.0178).
- **feature_3**: Consistently moderate across all methods (PFI: 0.0386, SHAP: 0.0129, LIME: 0.0281).
- **feature_8**: Consistently middle-ranked across all methods.

6.6 Interpretation for Business Stakeholders

"Our model uses 12 different measurements from the cheap sensors to predict the event type. There are sensors that directly measure the event type, but they're expensive, so we want to see if we can use the cheap sensors and a function so we don't need them. Because we have five different events that are each their own thing and don't overlap, we want to train a classifier model - a model that also predicts one of five hard classes. We want to train our model on data we already have, and then see if it can predict events by itself afterwards. Because we have way more data for one of the events, we're using a model that can focus more strongly on less common events. because otherwise, it might just always predict the most common class, because it's lazy. We trained a specific model - a so called random forest - because it's historically good with the type of problem we have. Now there are others, but we tested them against each other and the random forest performed the best. But, we know that our data has random noise - that's false data- as well. This data comes from sensor mistakes, but we have a number of measurements that we know are false. So next, we train an extra model, that tries to guess whether any data point is made by a sensor mistake. We then combine our models, to try and get a better predictor model, that also tells us whether something is a sensor mistake. We have eleven different measurements from our cheap sensors and we think, they can be used to predict which event happens. but, we don't know which of them are important. So we designed a test, that checks first how important each sensor measurement is to the model. The most important measurements are `feature_1`, `feature_0`, `feature_3`, and `feature_11` — these consistently influence predictions across multiple analysis methods.

So the analysis for our measurements we took are three types: permutation feature important - this just shuffles one measurement around and sees if the model gets a worse score. If it does, then that means the measurement is important. Next we looked at our measurements and how they each influence individual predictions. This tells us how important the measurement is to the model - but not how important it is in reality. We could expand on this to see if our model puts too much importance on irrelevant measurements too. Finally, because a random forest is not a model you can just look at and understand what's going on inside, we did a LIME analysis. that one tries to look at a small part of the model and figure out roughly what it does, then show it in a way that we can actually look at and understand.

Interestingly, some measurements that appeared visually important when looking at them at first (like `feature_10`) turned out to be not so important for the model's decisions. This means we could potentially simplify the sensor hardware by focusing on the few most informative measurements.

However, different analysis methods sometimes disagree, which is normal. This happens because features interact with each other — the model doesn't just look at each measurement in isolation, but considers combinations of measurements. The good news is that this confirms our model is learning meaningful patterns rather than just memorizing simple rules."